



中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

基于多元时序的分布式集群异常检测研究

作者姓名：杨婧雯

指导教师：

学位类别：硕士

学科专业：

培养单位：中国科学院大学计算机网络信息中心

2026 年 6 月

Research on Anomaly Detection for Distributed Clusters Based
on Multivariate Time Series

A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Master of
in
By
YANG Jingwen
Supervisor:

Computer Network Information Center, Chinese Academy of Sciences

June, 2026

中国科学院大学 学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。承诺除文中已经注明引用的内容外，本论文不包含任何其他个人或集体享有著作权的研究成果，未在以往任何学位申请中全部或部分提交。对本论文所涉及的研究工作做出贡献的其他个人或集体，均已在文中以明确方式标明或致谢。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日 期：

中国科学院大学 学位论文授权使用声明

本人完全了解并同意遵守中国科学院大学有关收集、保存和使用学位论文的规定，即中国科学院大学有权按照学术研究公开原则和保护知识产权的原则，保留并向国家指定或中国科学院指定机构送交学位论文的电子版和印刷版文件，且电子版与印刷版内容应完全相同，允许该论文被检索、查阅和借阅，公布本学位论文的全部或部分内容，可以采用扫描、影印、缩印等复制手段以及其他法律许可的方式保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：

日 期：

导师签名：

日 期：

摘 要

关键词：分布式集群，异常检测，多元时序，曲线相似性，自适应阈值

Abstract

Key Words: Distributed Cluster, Anomaly Detection, Multivariate Time Series, Curve Similarity, Adaptive Threshold

目 录

第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	2
1.3 论文研究内容和贡献	3
1.3.1 主要研究内容	3
1.3.2 主要贡献	3
1.4 论文组织结构	4
第 2 章 相关背景知识与理论基础	5
2.1 时间序列异常概述	5
2.1.1 时间序列异常数据简介	5
2.1.2 时间序列异常定义与分类	5
2.2 时序异常检测算法	6
2.2.1 单变量时序异常检测	6
2.2.2 多元时序异常检测	6
2.3 分布式集群监控数据与异常检测	7
2.3.1 分布式集群概述	7
2.3.2 集群监控数据特点	7
2.3.3 异常检测方法在集群数据中的应用与挑战	8
2.4 本章小结	8
第 3 章 面向多节点单指标的异常检测算法	9
3.1 多节点单指标数据集构建	9
3.1.1 数据来源与采集	9
3.1.2 数据集预处理与标注	9
3.1.3 数据集统计与可视化	10
3.2 现有异常检测方法评估	10
3.2.1 异常检测基线算法	11
3.2.2 基于曲线相似性的异常检测方法	11
3.2.3 实验结果对比分析	13
3.3 参数敏感性分析与优化策略	13
3.3.1 阈值敏感性实验与分析	13
3.3.2 参数依赖性与泛化挑战	15

3.4 基于极值理论的自适应阈值算法	15
3.4.1 经典 SPOT 算法原理与局限	15
3.4.2 包容性 SPOT 算法的改进策略	15
3.4.3 集成学习框架设计与实现	16
3.5 实验验证与对比分析	16
3.5.1 实验设置	16
3.5.2 与基线算法实验结果对比	17
3.5.3 阈值稳定性与自适应效果分析	17
3.6 本章小结	17
第 4 章 数值-形状双视角融合的多指标多节点异常检测	19
4.1 集群多指标多节点数据集及异常检测算法框架	19
4.1.1 多指标多节点数据集概述	19
4.1.2 问题分析与算法框架	20
4.2 基于数值特征的异常检测算法	21
4.2.1 多指标距离度量方法	21
4.2.2 加权融合设计	22
4.2.3 算法效率优化	22
4.3 基于形状特征的异常检测算法	23
4.3.1 经典 SAX 算法原理	23
4.3.2 基于 SAX 算法的符号化表示与模式匹配	23
4.3.3 算法效率优化	24
4.4 多算法融合与故障定位	24
4.4.1 加权融合设计	24
4.4.2 故障定位与根因分析建议	25
4.5 实验验证与对比分析	25
4.5.1 实验设置	25
4.5.2 方法结果对比分析与案例研究	26
4.5.3 融合方法优势分析	26
4.5.4 算法效率实验对比	26
4.6 本章小结	27
第 5 章 基于曲线相似性的异常检测系统	29
5.1 总体流程设计	29
5.2 系统架构与关键技术	29
5.3 系统实现	30
5.4 系统应用效果展示	30
5.5 本章小结	31

第 6 章 总结与展望	33
6.1 本文工作总结	33
6.2 未来研究展望	33
参考文献	35
附录一 附录中的公式	37
附录二 附录中的图表	39
致谢	41
作者简历及攻读学位期间发表的学术论文与其他相关学术成果 ..	43

图目录

图 3-1 测试数据集异常标注曲线（红色阴影部分为异常标注）	10
附图 2-1 这是一个样图	39

表目录

表 3-1 数据集统计数据	10
表 3-2 基线算法对单指标多节点的 RPC Router 连接数异常检测效果	13
表 3-3 欧氏距离方法在不同阈值和时间窗口下的异常检测分数	14
表 3-4 AE 嵌入后异常检测分数	14
表 3-5 DBSCAN 异常检测分数	14
表 3-6 集成学习算法分数	16
表 3-7 SPOT 算法性能对比	17
表 4-1 多指标多节点异常检测方法对比	26
表 4-2 算法效率对比	26
附表 2-1 这是一个样表	39
附表 2-2 这是一个样表	39

符号列表

字符

Symbol	Description	Unit
R	the gas constant	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_v	specific heat capacity at constant volume	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_p	specific heat capacity at constant pressure	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
E	specific total energy	$\text{m}^2 \cdot \text{s}^{-2}$
e	specific internal energy	$\text{m}^2 \cdot \text{s}^{-2}$
h_T	specific total enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
h	specific enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
k	thermal conductivity	$\text{kg} \cdot \text{m} \cdot \text{s}^{-3} \cdot \text{K}^{-1}$
S_{ij}	deviatoric stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
τ_{ij}	viscous stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
δ_{ij}	Kronecker tensor	1
I_{ij}	identity tensor	1

算子

Symbol	Description
Δ	difference
∇	gradient operator
$\delta^\pm$	upwind-biased interpolation scheme

缩写

CFD	Computational Fluid Dynamics
CFL	Courant-Friedrichs-Lewy
EOS	Equation of State
JWL	Jones-Wilkins-Lee
WENO	Weighted Essentially Non-oscillatory
ZND	Zel'dovich-von Neumann-Doering

第 1 章 绪论

1.1 研究背景及意义

分布式集群（如 Hadoop, Kubernetes）是结合了分布式系统和集群系统优点的一种架构，在分布式集群中，不同的服务器负责不同的任务，同时每个任务可以由多个服务器组成的集群来处理。分布式集群是支撑现代云计算与大数据处理的核心基础设施，其稳定性直接关系到上层业务的连续性。此类集群在运行中会产生海量的多维度监控时序数据（如 CPU 负载、内存使用等），这些数据是感知系统健康状态、诊断异常的关键。

然而，随着集群规模不断扩大、架构动态性增强以及业务负载波动加剧，传统的基于阈值的异常检测方法因依赖先验知识、适应性差，导致误报和漏报率高，难以满足生产环境需求。尽管基于深度学习的方法能自动学习复杂模式，但在实际生产环境中，一个核心挑战在于对于如 RPC 连接数这类与用户任务相关的指标，其“正常”的绝对范围和模式难以统一定义，会随系统整体负载发生无规律波动，若使用基于历史数据训练的模型，会在整体高负载时产生大量误报，严重干扰运维效率。在分布式集群中，由于负载均衡机制的作用，在系统正常运行时，同一时刻各节点所承受的负载是相近的，各个节点指标曲线往往表现出极大的相似性，在运维实践中，运维人员常通过对比分布式集群中的同一指标曲线来判断异常：与其他变化模式相差很大的“离群曲线”更可能是异常。本研究正是基于这一洞察，旨在将“分布式集群相似性”这一先验知识系统性地引入多元时序异常检测模型，通过对比集群内多个节点在同一监控指标上的行为曲线，来识别其中显著偏离“群体一致性”模式的个别节点或子集，以提升异常检测的效果。

具体而言，本研究针对的是多节点时序数据集：即对同一指标（如 CPU 使用率），采集自集群中多个节点的、具有时间对齐关系的多条时序曲线。该数据集不仅包含每个节点自身的时序变化模式，更隐含了节点之间的运行关联与协同规律。如何有效捕捉并利用这种分布式关联，构建能够区分节点个体异常与集群整体波动的检测模型，是本研究要解决的关键问题。

本研究的意义在于，从方法和实践应用两个层面为分布式集群异常检测提供了新的思路与解决方案。在方法层面，本研究跳出了传统异常检测聚焦于“时间”与“指标”二维分析的框架，创新性地将“集群节点”这一空间维度系统性地引入模型，构建了“时间 $\times$ 指标 $\times$ 节点”的三维分析范式。这一范式将集群内在的、由负载均衡机制保证的节点行为相似性，从一种隐性的运维经验转化为一种显式的、可计算的先验知识，为处理动态负载场景下的异常检测问题提供了新的视角。在实践应用层面，所提出的方法直接针对生产环境中负载敏感型指标检测的痛点，通过识别“离群曲线”而非定义“绝对正常”，能够有效降低因集群整体负载波动引发的误报，提升检测结果的可靠性。基于真实运维数据构建数据集并

验证算法,既保证了方法在实际业务场景下的有效性,也为后续研究提供了数据基础。本研究结果不仅有助于实现更精准的故障定位与预警,缩短平均故障恢复时间,提升业务连续性,也为提升真实集群运维体系智能化提供了核心的检测能力支撑,具有重要的工程应用价值。

1.2 国内外研究现状

多元时间序列异常检测的研究历程呈现出清晰的方法论演进脉络。早期研究主要依赖于基于传统统计学的方法,例如基于统计阈值的方法依赖于统计学原理来识别数据中的异常模式,或采用时域和频域分析方法通过傅里叶变换与小波变换分析时序数据的频率信息,或采用自回归差分移动平均模型等时间序列分解技术。此类方法虽具有原理简单、可解释性强的优点,但在处理高维且蕴含复杂非线性关联的多元时序数据时往往能力有限,同时对数据噪声和振幅变化较为敏感。

随后,基于机器学习的方法逐渐得到应用,如孤立森林、支持向量机等算法通过将时序数据转换为特征向量进行异常识别。然而,这类方法的效果在很大程度上依赖于精细的特征工程,需要较多的专家知识,且同样难以有效捕捉变量间复杂的时空依赖关系。近年来,基于深度学习的方法已成为该领域的主流研究方向。此类方法能够自动学习数据的高层表征,显著提升了对复杂模式的刻画能力。当前深度学习方法主要可分为两大范式。其一是基于重构的模型,以自编码器及其变体为代表,其核心假设是模型在学习了正常数据分布后难以对异常样本进行准确重构,从而通过较大的重构误差来识别异常,代表性工作如 OmniAnomaly^[1] 和 USAD^[2] 等算法。其二是基于预测的模型,利用长短期记忆网络、Transformer 等序列模型来预测未来时间步的取值,并将预测值与真实值之间的偏差作为异常分数,这类方法在捕捉时序依赖性方面展现出优势。此外,前沿探索开始引入图神经网络来显式建模监控指标间的拓扑关系,代表性工作如 GDN^[3] 通过学习传感器之间的图结构来检测异常,而 Transformer 架构因其强大的长程依赖捕捉能力也成为研究热点,代表性工作如 TranAD^[4] 和 Anomaly Transformer^[5] 等。

综上所述,现有研究虽然在多元时序异常检测方面取得了显著进展,但绝大多数工作仍聚焦于时间与指标这两个维度构成的二维数据分析框架内。一个普遍的局限性在于,未能系统性地挖掘和利用分布式集群生产环境中广泛存在的多节点这一宝贵维度信息。换言之,当前方法未能充分利用分布式集群中天然存在的指标行为模式的相似性来辅助判别概念漂移现象与真实异常。因此,本研究旨在填补这一空白,探索一种基于时间、指标与集群节点的三维数据分析新范式,以期提升异常检测在真实动态运维场景中的鲁棒性。

1.3 论文研究内容和贡献

1.3.1 主要研究内容

本文围绕分布式集群多节点场景下的时序异常检测问题，开展了以下三个方面的研究工作。

第一，面向单指标多节点场景的异常检测算法研究。本文以 YARN 集群的 RPC Router 连接数指标为研究对象，构建了基于联通大数据生产环境的单指标多节点时序数据集。在此基础上，分别采用欧氏距离、自编码器嵌入距离和 DBSCAN 聚类三种方法度量节点曲线间的相似性，验证了基于群体一致性思想的异常检测方法的有效性。针对单一方法阈值敏感的问题，设计了基于逻辑与策略的集成学习框架，并引入改进的 SPOT 算法实现阈值的动态自适应调整。

第二，面向多指标多节点场景的异常检测算法研究。本文针对 GPU 集群故障数据集，设计了数值与形状双视角融合的异常检测框架。在数值视角方面，采用基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面，提出了改进的 SAX 符号化表示算法，通过取消分段聚合保留原始分辨率并采用严格距离计算规则来提升对形状变化的敏感性。最终通过加权融合两种视角的检测结果，实现对不同类型异常的全面覆盖。

第三，基于曲线相似性的异常检测系统设计与实现。本文设计了面向分布式集群的异常检测系统架构，实现了从数据采集、预处理、异常检测到结果展示的完整流程，并通过算法效率优化保障了系统在大规模集群环境下的实时性要求。

1.3.2 主要贡献

本文的主要贡献包括以下三个方面。

第一，本文提出了一种基于多节点曲线相似性的分布式集群异常检测方法。该方法突破了传统异常检测聚焦于时间与指标二维分析的框架，创新性地 will 集群节点这一空间维度系统性地引入模型，构建了时间、指标与节点的三维分析范式。通过利用分布式集群负载均衡机制所保证的节点行为相似性，将隐性的运维经验转化为显式的可计算先验知识。

第二，本文设计了数值与形状双视角融合的多指标异常检测框架。该框架从数值幅度和曲线形态两个互补角度刻画节点异常行为，其中改进的 SAX 算法通过保留原始分辨率和采用严格距离计算规则增强了对形状变化的敏感性。通过加权融合两种视角的检测结果，该框架能够有效识别不同类型的节点异常。

第三，本文基于真实生产环境构建了单指标多节点时序数据集和多指标多节点故障数据集，并在这些数据集上验证了所提方法的有效性。实验结果表明，本文所提方法在检测准确率方面显著优于现有的多元时序异常检测算法，为分布式集群异常检测提供了新的解决方案。

1.4 论文组织结构

本文共分为六章，各章内容安排如下。

第一章绪论部分阐述了本研究的背景与意义，回顾了多元时间序列异常检测领域的国内外研究现状，概述了本文的主要研究内容与贡献。

第二章介绍了相关背景知识与理论基础，包括时间序列异常的定义与分类、主流的时序异常检测算法，以及分布式集群监控数据的特点与挑战。

第三章针对单指标多节点场景，详细介绍了数据集的构建过程、基于曲线相似性的异常检测方法，以及集成学习和自适应阈值等改进策略。

第四章针对多指标多节点场景，提出了数值与形状双视角融合的异常检测框架，详细阐述了基于欧氏距离的数值异常检测算法和基于改进 SAX 的形状异常检测算法，并介绍了多算法融合与故障定位方法。

第五章介绍了基于曲线相似性的异常检测系统，包括系统的总体流程设计、架构与关键技术、实现细节及应用效果展示。

第六章对全文工作进行总结，并对未来研究方向进行展望。

第2章 相关背景知识与理论基础

2.1 时间序列异常概述

时间序列数据作为记录系统、过程或现象随时间演变信息的重要载体，广泛存在于工业监控、金融交易、网络安全及物联网等众多领域。对时间序列中的异常进行准确、及时的检测，是感知系统状态、预测潜在故障、保障业务稳定的关键技术。本节将系统阐述时间序列异常检测的定义、分类等核心概念。

2.1.1 时间序列异常数据简介

时间序列数据是按照时间顺序排列的一系列观测值的集合，其数学形式可表示为 $\{x_1, x_2, \dots, x_T\}$ ，其中 x_t 表示在时刻 t 的观测值， T 为序列长度。在单变量时间序列中，每个时刻的观测值为标量；在多元时间序列中，每个时刻的观测值为向量 $\mathbf{x}_t \in \mathbb{R}^M$ ，其中 M 表示变量维度。

时间序列数据具有若干典型特征。首先是时序相关性，即相邻时刻的观测值之间往往存在较强的相关性，这种相关性可以是短期的自相关，也可以是长期的依赖关系。其次是周期性和季节性，许多时间序列数据呈现出周期性波动的规律，例如网络流量的日周期和周周期变化。第三是趋势性，时间序列可能呈现长期的上升或下降趋势。第四是非平稳性，时间序列的统计特性可能随时间发生变化，表现为均值、方差或相关结构的漂移。

在分布式集群监控场景中，时间序列数据通常以固定采样间隔连续采集，涵盖 CPU 使用率、内存占用、网络流量、磁盘读写等多个维度。这些监控指标共同反映了系统的运行状态，为故障诊断和性能优化提供了重要的数据基础。

2.1.2 时间序列异常定义与分类

时间序列中的异常是指与数据的正常行为模式存在显著偏离的观测点或观测片段。根据异常的时空范围和表现形式，可将时间序列异常划分为以下几类。

第一类是点异常，指单个时刻的观测值显著偏离其正常范围。点异常可进一步分为全局点异常和上下文点异常。全局点异常是指观测值在整个序列范围内都属于异常，例如数值突然飙升至历史最高点。上下文点异常是指观测值虽然在全局范围内并不极端，但在其局部上下文中显得异常，例如在夜间低峰期出现的中等流量峰值。

第二类是模式异常，指时间序列在某一时间段内呈现出异常的变化模式。模式异常强调的是曲线形态的异常，而非单个数值的异常。例如，原本平稳的指标突然出现剧烈震荡，或者周期性指标的周期规律被打破。

第三类是集体异常，指多个观测点共同构成的异常行为，其中单个观测点可能并不异常，但作为一个整体则表现出异常特征。例如，某段时间内的观测值虽然都处于正常范围，但其整体趋势或分布与历史规律不符。

在多元时间序列场景中，异常还可根据其影响范围分为单变量异常和多变量关联异常。单变量异常仅涉及单个监控指标，而多变量关联异常涉及多个指标之间的异常关联或依赖关系的破坏。

在分布式集群场景中，本文重点关注的是节点级异常，即在多节点环境下，个别节点的监控曲线偏离集群整体行为模式的情形。这类异常的识别需要同时考虑时间维度的变化规律和节点维度的群体一致性。

2.2 时序异常检测算法

2.2.1 单变量时序异常检测

单变量时序异常检测旨在识别单条时间序列中的异常点或异常片段，其方法体系经历了从统计方法到机器学习方法再到深度学习方法的演进。

传统统计方法主要包括基于阈值的方法和基于时序模型的方法。基于阈值的方法通过设定上下界来判断观测值是否异常，阈值可以是固定值，也可以根据历史数据的统计特性动态计算，例如采用均值加减若干倍标准差作为异常边界。基于时序模型的方法则利用自回归模型、移动平均模型及其组合形式对时间序列进行建模，通过观测值与模型预测值之间的残差来识别异常。这类方法的优点是原理简单、计算效率高，但其假设时序数据服从特定的统计分布，在面对复杂非线性模式时效果有限。

机器学习方法通过将时间序列转换为特征向量来进行异常检测。常用方法包括孤立森林，该方法通过随机划分数空间来隔离异常点，异常点由于偏离正常数据分布而更容易被隔离。另一类方法是单类支持向量机，该方法学习正常数据的边界，将落在边界之外的点判定为异常。这类方法需要精心设计特征表示，其效果依赖于特征工程的质量。

深度学习方法通过自动学习时序数据的表征来进行异常检测。自编码器是一种典型方法，它学习将正常数据压缩为低维表示并重构，对于异常数据由于未在训练中见过而产生较大的重构误差。循环神经网络和长短期记忆网络能够捕捉时序数据的长期依赖关系，常用于预测下一时刻的取值并通过预测误差识别异常。

2.2.2 多元时序异常检测

多元时序异常检测需要同时处理多个监控指标，不仅要识别单个指标的异常，还要捕捉指标之间的异常关联。

基于重构的方法是多元时序异常检测的主流范式之一。OmniAnomaly 算法采用随机变分自编码器和平面归一化流来学习多元时间序列的正常模式分布，通过重构概率识别异常。USAD 算法基于对抗性训练的自编码器架构，通过引入对抗性训练来增强自编码器的鲁棒性，从而提升重构误差对异常的敏感性。

基于预测的方法通过预测未来时间步的取值并将预测误差作为异常分数。

TranAD 算法基于 Transformer 架构, 利用注意力机制捕捉长期依赖, 并通过对抗训练增强对异常的敏感度。MTAD-GAT 算法结合了图注意力网络和时间卷积网络, 同时建模变量关联和时间依赖。

基于图神经网络的方法能够显式建模变量间的拓扑关系。GDN 算法学习传感器之间的图结构, 通过图神经网络传播信息并预测未来值, 能够捕捉变量之间的复杂依赖关系。

基于生成对抗网络的方法利用生成器和判别器的对抗训练来学习正常数据分布。MAD-GAN 算法基于 LSTM 的生成对抗网络, 通过判别器得分和重构误差共同识别异常。

上述方法虽然在多元时序异常检测方面取得了显著进展, 但主要关注时间维度和指标维度, 未能充分利用分布式集群中多节点之间的行为相似性。本文的研究正是旨在弥补这一不足, 将节点维度纳入异常检测的分析框架。

2.3 分布式集群监控数据与异常检测

2.3.1 分布式集群概述

分布式集群是结合了分布式系统和集群系统优点的一种计算架构。在分布式集群中, 不同的服务器节点负责不同的任务, 同时每个任务可以由多个服务器组成的集群来协同处理。典型的分布式集群架构包括 Hadoop 生态系统中的 YARN 集群和容器编排领域的 Kubernetes 集群等。

以 YARN 集群为例, 其架构由资源管理器和多个节点管理器组成。资源管理器负责整个集群的资源调度和管理, 节点管理器运行在集群的各个计算节点上, 负责管理节点的资源和执行容器。当用户提交计算任务时, 资源管理器根据资源需求和调度策略将任务分配到合适的节点上执行。为保证系统的可扩展性和容错性, YARN 集群通常采用负载均衡机制, 使得计算任务能够均匀分布到各个节点上。

分布式集群是支撑现代云计算与大数据处理的核心基础设施, 广泛应用于数据仓库、实时计算、机器学习训练等场景。此类集群的稳定运行直接关系到上层业务的连续性, 因此对集群的监控和异常检测具有重要的实际意义。

2.3.2 集群监控数据特点

分布式集群在运行过程中会产生海量的多维度监控时序数据, 这些数据具有若干显著特点。

首先是多维度性。集群监控涵盖多个层次和维度, 包括系统级指标如 CPU 使用率和内存占用, 应用级指标如请求响应时间和连接数, 以及集群级指标如任务队列长度和资源使用率等。这些指标从不同角度反映系统的健康状态。

其次是高频采样性。为及时发现异常, 监控系统通常以较高频率采集数据, 常见的采样间隔为数秒到数分钟。高频采样产生的数据量巨大, 对存储和处理提

出了较高要求。

第三是多节点相关性。在分布式集群中，由于负载均衡机制的作用，各节点在正常运行时承担的负载相近，其监控指标曲线往往表现出高度的形态相似性。这种相似性为基于群体一致性的异常检测提供了天然的基础。

第四是动态波动性。与用户负载相关的指标具有正常模式难以统一定义的特点，其数值会随系统整体负载发生无规律波动。这种动态性增加了异常检测的难度，使得基于固定阈值或历史模式学习的方法容易产生误报。

2.3.3 异常检测方法在集群数据中的应用与挑战

将现有异常检测方法应用于分布式集群监控数据时面临若干挑战。

第一个挑战是正常模式的定义困难。传统方法假设存在明确的正常模式基线，但对于负载敏感型指标，其正常范围会随集群整体负载动态变化。采用基于历史数据训练的模型进行检测，容易在整体高负载时产生大量误报。

第二个挑战是概念漂移问题。分布式集群的运行模式可能随业务变化、版本升级或配置调整而发生变化，导致历史学习到的模式不再适用于当前数据。传统方法难以区分这种正常的模式演变与真实的异常。

第三个挑战是多节点协同分析的缺失。现有多元时序异常检测方法主要关注时间维度和指标维度，未能有效利用集群中多节点之间的行为相似性。这种相似性蕴含了丰富的先验知识，能够为异常检测提供重要的参考依据。

针对上述挑战，本文提出利用分布式集群多节点曲线的相似性进行异常检测。核心思想是，在负载均衡机制下，正常节点的曲线应保持较高的相似性，而异常节点的曲线会显著偏离群体行为模式。通过对比节点间的曲线差异，能够有效识别离群的异常节点，避免因定义绝对正常而产生的误报问题。

2.4 本章小结

本章系统介绍了时间序列异常检测的相关背景知识与理论基础。首先阐述了时间序列异常数据的基本概念和特征，并对异常类型进行了分类。其次回顾了单变量和多元时序异常检测的主流方法，涵盖了从统计方法到深度学习方法的发展脉络。最后介绍了分布式集群的基本概念和监控数据特点，分析了现有异常检测方法在集群场景中应用时面临的挑战，为后续章节的算法设计奠定了理论基础。

第3章 面向多节点单指标的异常检测算法

3.1 多节点单指标数据集构建

本章研究的数据基础来源于真实的分布式集群生产环境，旨在构建能够反映集群节点协同行为、并包含明确异常标注的数据集，以支撑后续算法的训练、验证与对比分析。

3.1.1 数据来源与采集

本研究的数据来源于中国联通的大数据生产底座平台。该平台采用标准的 YARN (Yet Another Resource Negotiator) 架构管理分布式计算资源，其运行状态通过 Prometheus 监控系统进行全方位、多粒度的采集与存储。

在运维实践中发现，与用户负载强相关的指标（如 RPC 连接数）具有正常模式难以统一定义、易随全局负载无规律波动的特点，导致传统基于绝对阈值或历史模式学习的检测方法误报率高。然而，得益于分布式集群的负载均衡设计，在系统正常运行时，同一时刻集群内各节点承担的负载相近，其同一监控指标曲线表现出高度的形态相似性。这一现象为基于“群体一致性”的异常检测提供了天然的基线。

基于此洞察，本研究选定“RPC Router 连接数”作为核心指标。该指标直接反映了数据节点与资源管理器 (ResourceManager) 之间的通信负载与健康状态，是感知节点服务能力的关键信号。我们通过 Prometheus API，采集了多个生产集群在连续时间段内（训练集：30 天；测试集：8 天）所有节点的该指标时序数据，原始采样间隔为 60 秒。

3.1.2 数据集预处理与标注

为构建适用于算法研究的高质量数据集，并对后续模型性能进行可靠评估，本研究对从生产环境采集的原始监控时序数据实施了系统化的预处理流程，并采用了半人工标注策略。

原始监控数据在采集与传输过程中，可能因网络延迟、数据采集器 (exporter) 瞬时故障或系统抖动而产生缺失值。为确保时序数据的连续性，本研究采用线性插值法对短暂的数据缺失点进行填充。为确保不同节点间的时序曲线在时间维度上严格对齐，以进行有效的跨节点相似性比较，我们将所有数据点对齐至统一的 60 秒间隔时间戳上。此外，对于因监控代理长期离线、节点重启或下线等原因导致的长时间段数据完全缺失，线性插值将引入较大误差，针对此类情况，本研究直接剔除包含长时间段数据缺失的整条节点记录，以保证进入模型的数据的完整性与可靠性。

获得高质量的异常标注是进行模型评估和算法对比的基础。然而，生产环境中的异常样本缺乏记录，且缺乏自动化标注工具。本研究采用“算法初步筛选 +

专家经验判读”相结合的方式进行半人工标注，旨在确保标注结果的准确性。具体流程如下：

首先，利用多种基础的无监督离群点检测算法对全体时序数据进行初步扫描，且将阈值设为宽松值，筛选出偏离其自身历史基线或群体平均模式的”疑似异常”时间片段。此步骤旨在缩小人工审查的范围，提升标注效率。

其次，对于初筛出的疑似异常片段，关联查询该时段内对应节点的运维数据，如日志、运维工单记录、相关指标时序数据等，对初步筛选出的异常点进行交叉验证，辅助判断其是否为真实的节点级故障或性能异常。

最终，由具备丰富分布式集群运维经验的专家，结合上一步的关联数据，并可视化审查疑似片段及其前后上下文的多节点曲线对比图，最终做出异常点判定。通过以上流程，对两个独立的 YARN 集群测试集进行了标注，形成了包含精确起止时间戳的异常片段集合。

3.1.3 数据集统计与可视化

经过预处理与标注，本研究构建了一个适用于分布式集群节点级异常检测研究的单指标多节点时间序列数据集。该数据集的详细统计信息如表3-1所示。

表 3-1 数据集统计数据

	节点数	数据点数量	异常片段数	异常点数量	异常比例
训练集	30	86400	/	/	/
测试集 A	15	11520	72	901	7.82%
测试集 B	15	11520	33	907	7.87%
测试集	30	23040	105	1808	7.85%

本数据集的核心特征在于其直观揭示了分布式集群在负载均衡机制下的运行规律，如图3-1所示。在绝大部分时间内，集群内多数节点的监控指标曲线呈现出高度的形态同步性与数值聚集性，它们紧密缠绕，形成清晰的”正常行为簇”。这反映了在系统负载均衡策略下，各节点承担相近的工作负载，行为模式高度一致。然而，当个别节点发生资源竞争、进程僵死、网络隔离或本地硬件故障等问题时，其监控曲线会显著偏离这个共同的”行为簇”，表现为形态各异、幅度不一的”离群曲线”。这种”正常聚集，异常离群”的鲜明数据特征，是本研究所提出的、基于群体相似性进行异常检测方法的依据与直观体现。

图 3-1 测试数据集异常标注曲线（红色阴影部分为异常标注）

3.2 现有异常检测方法评估

本节首先系统评估了一系列现有的时间序列异常检测算法在本研究构建的分布式集群数据集上的性能。3.2.1节与3.2.2节分别介绍了选择的先进的异常检

测基线算法和基于曲线相似性的检测方法。在3.2.3节介绍了以上算法在分布式集群数据集上的实验结果。这些算法代表了不同的主流范式，其评估结果不仅揭示了现有方法在处理“多节点相似性”场景时的共性局限，也为后续提出的新方法提供了性能对比的锚点。

3.2.1 异常检测基线算法

我们选取了六种具有代表性的先进异常检测算法作为基线进行对比，涵盖基于重构、预测及生成对抗网络等多种主流范式：

基于重构的模型：

- **OmniAnomaly**：一种基于随机变分自编码器（VAE）和平面归一化流的算法，旨在学习多元时间序列的正常模式分布，并通过重构概率识别异常。
- **USAD**：基于对抗性训练的自编码器模型，通过引入对抗性训练来增强自编码器的鲁棒性，从而提升重构误差对异常的敏感性。

基于预测的模型：

- **GDN**：一种基于图神经网络的异常检测模型，显式建模变量间的拓扑关系，并通过预测未来值来检测异常。
- **TranAD**：基于 Transformer 架构的深度异常检测模型，利用注意力机制捕捉长期依赖，并通过对抗训练增强对异常的敏感度。
- **MTAD-GAT**：一种多角度时序异常检测模型，结合了图注意力网络和时间卷积网络，同时建模变量关联和时间依赖。

基于生成对抗网络（GAN）的模型：

- **MAD-GAN**：基于 LSTM 的生成对抗网络，利用生成器和判别器的对抗训练来学习正常数据分布，并通过判别器得分和重构误差识别异常。

实验设置遵循公平对比原则。所有基线模型均使用其作者开源的原版代码和默认推荐参数进行训练与测试。训练集使用??节所述的同构集群 30 天正常数据。测试则在本研究标注的两个测试集（Test A, Test B）上进行，并报告合并测试集的整体性能。评估指标采用异常检测领域通用的精确率（Precision）、召回率（Recall）和 F1 分数（F1-Score）。

3.2.2 基于曲线相似性的异常检测方法

3.2.2.1 基于数值的相似性度量

欧氏距离是最直观的数值差异度量方式，用于直接衡量节点间曲线在数值空间中的差异，其基本定义如下：给定一个包含 N 个节点的时间序列集合，每个节点在时刻 t 的观测值为 x_i^t (i 为节点索引)，为捕捉时序局部模式，首先采用滑动窗口对序列进行分段处理。对于长度为 w 的窗口，节点 i 在时刻 t 的窗口序列可表示为向量 $\mathbf{x}_i^t = [x_i^{t-w+1}, \dots, x_i^t]$ ，节点 i 与节点 j 在时刻 t 的欧氏距离定义为两节点窗口向量之间的 L2 范数：

$$d_{ij}^t = \|\mathbf{x}_i^t - \mathbf{x}_j^t\|_2 = \sqrt{\sum_{k=t-w+1}^t (x_i^k - x_j^k)^2} \quad (3-1)$$

为识别多节点中的异常，首先在时刻 t 的滑动窗口内，计算所有节点两两之间的欧氏距离，并求全局平均值 μ_t 和标准差 σ_t ，以此表征当前滑动窗口内整个集群的曲线整体相似性水平；接着，逐一计算节点 i 在时刻 t 与其他所有同集群节点（共 $N - 1$ 个）的欧氏距离的平均值 $\bar{d}_i^t$ ，节点 i 在时刻 t 的异常分数为基于 Z-score 的标准化形式：

$$s_i^t = \frac{\bar{d}_i^t - \mu_t}{\sigma_t} \quad (3-2)$$

判断异常时，设定阈值 T ，若 $s_i^t > T$ ，则节点 i 在时刻 t 标记为异常。欧氏距离方法基于以下假设：在负载均衡场景下，正常节点的曲线应保持较高相似性，其异常分数将聚集在较低水平；而异常节点的曲线会显著偏离其他节点，导致其异常分数明显偏高。

3.2.2.2 基于形态的相似性度量

欧氏距离对曲线的绝对数值敏感，但可能忽略了形状的相似性。例如，两条相位相同、振幅不同的周期性曲线，其欧氏距离可能很大，但形态一致。为此，我们引入基于自编码器的深度特征提取方法，学习曲线形态的高层抽象表示。

对曲线在形态上的相似性捕捉不足，本研究引入自编码器进行特征嵌入，旨在将原始时序曲线映射到更能反映其潜在形态特征的低维空间。

自编码器是一种无监督神经网络，其目标是学习一个高效的数据表示。它由编码器（Encoder）和解码器（Decoder）两部分组成。编码器将高维输入数据（如一个时间窗口的曲线）压缩为低维的嵌入向量（embedding vector），这个向量捕获了输入数据中最显著的特征模式，解码器则尝试从这个低维嵌入向量中重构出原始输入。通过训练使重构误差最小化，AE 迫使嵌入向量能够保留关于输入曲线形态的关键信息。

首先对自编码器进行训练，训练方法是让模型重构输入的时间序列窗口，最小化 MSE（均方误差）。AE 训练完成后，使用编码器为每个节点的时间窗口序列生成对应的嵌入向量，后续基于嵌入向量间的平均欧氏距离计算异常分数，后续计算与阈值判断流程与前述欧氏距离方法一致。

3.2.2.3 基于密度的相似性度量

从非统计度量的角度，基于密度的聚类算法 DBSCAN 也能够用于异常离群点检测。DBSCAN 是一种经典的无监督聚类算法，它将数据点划分为高密度区域对应的簇，并能够识别位于低密度区域的离群点。该方法的核心优势在于无需预先指定簇的数量，且能将离群点自动识别为噪声。

使用 DBSCAN 聚类算法对每个时间窗口内的曲线进行聚类，被标记为噪声点的节点即被判定为在该时刻发生异常。其基本逻辑是：在负载均衡场景下，大多数正常节点的曲线形态相似，应在特征空间中聚集形成高密度簇；而行为异常的节点其曲线会显著偏离主流模式，从而落在密度较低的区域，被 DBSCAN 识别为噪声。

3.2.3 实验结果对比分析

现有异常检测算法在单指标多节点 RPC 连接数数据集上的性能表现如表3-2所示。

表 3-2 基线算法对单指标多节点的 RPC Router 连接数异常检测效果

算法	Precision	Recall	F1
TranAD	0.8889	0.3932	0.5452
USAD	0.6892	0.5567	0.6159
OmniAnomaly	0.6054	0.4593	0.5224
GDN	0.3101	0.5218	0.3890
MAD_GAN	0.6309	0.6309	0.6891
MTAD_GAT	0.3492	0.1573	0.2169
欧氏距离	0.9826	0.9657	0.9741
AE	0.9792	0.9592	0.9691
DBSCAN	0.9605	0.9659	0.9632

对负载均衡类指标，波动具有随机性，不易学习正常模式，当前的预测和重构类算法往往只能学习到主要模式，对于离群值和大幅波动易漏报误报，在此类任务上的表现一般。如表3-2所示，传统多指标异常检测算法对单指标多节点的 RPC Router 连接数数据集能够检测出单个节点的显著波动，但对于单个节点显著离群的非显著波动检测效果较差。

欧氏距离、自编码器嵌入距离及 DBSCAN 三种基于曲线相似性的算法在阈值参数设置得当时，均能取得较高的异常检测 F1 分数，证明了基于多节点曲线相似度比较的核心思路是有效的。然而，这些方法的性能对阈值选择极为敏感，阈值设置的细微偏差便会导致检测效果急剧下降，下一节将详细介绍参数敏感性实验及分析。

3.3 参数敏感性分析与优化策略

3.3.1 阈值敏感性实验与分析

在不同窗口大小和阈值上对基于曲线相似性的欧氏距离、AE、DBSCAN 异常检测算法进行实验，结果如表3-3、表3-4和表3-5所示。

表 3-3 欧氏距离方法在不同阈值和时间窗口下的异常检测分数

threshold	window size=10			window size=20			window size=30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.5485	1.0000	0.7084	0.7251	1.0000	0.8406	0.8214	1.0000	0.9020
3.2	0.6797	1.0000	0.8093	0.7892	1.0000	0.8822	0.8593	1.0000	0.9243
3.4	0.8116	1.0000	0.8960	0.8974	1.0000	0.9459	0.9256	1.0000	0.9614
3.6	0.9532	0.9655	0.9593	0.9826	0.9657	0.9741	0.9727	0.9469	0.9596
3.7	0.9946	0.9242	0.9581	0.9928	0.8839	0.9352	0.9944	0.8856	0.9368
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

表 3-4 AE 嵌入后异常检测分数

threshold	window size=10			window size=20			window size=30		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
3.0	0.4370	0.9833	0.6051	0.6028	1.0000	0.7522	0.7206	1.0000	0.8376
3.6	0.9507	0.8977	0.9234	0.9793	0.9404	0.9595	0.9792	0.9592	0.9691
3.8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

表 3-5 DBSCAN 异常检测分数

eps	Precision	Recall	F1
200	0.0332	1.0000	0.0642
300	0.3442	1.0000	0.5121
400	0.8302	1.0000	0.9072
500	0.9605	0.9659	0.9632

实验结果显示,基于欧氏距离的检测方法在窗口大小为 20、阈值为 3.6 时取得了最佳 F1 分数。然而,该方法对阈值选择极为敏感,例如当阈值从 3.6 略微增加至 3.7 时,召回率出现显著下降,而阈值升至 3.8 时,检测完全失效 ($F1=0$)。AE 方法窗口大小为 30,阈值为 3.6 时取得最佳 F1 分数,但同样面临阈值选择敏感问题,当阈值升至 3.8 时,检测完全失效。DBSCAN 算法通过调整邻域半径 ϵ ,能够有效识别异常,在 $\epsilon=500$ 时取得了最佳 F1 分数,但仍呈现出算法效果随 ϵ 参数设定波动明显的特点。

3.3.2 参数依赖性与泛化挑战

以上实验表明,欧氏距离、自编码器嵌入距离及 DBSCAN 三种基础算法在阈值参数设置得当时,均能取得较高的异常检测 F1 分数,证明了基于多节点曲线相似度比较的核心思路是有效的。然而,这些方法的性能对阈值选择极为敏感,阈值设置的细微偏差便会导致检测效果急剧下降。最优阈值的确定严重依赖于已标记的验证数据集,但在实际生产环境中,获取大量高质量的标记数据并不现实,且指标的异常通常极少或没有,使得针对每个场景或指标进行人工阈值标定成本高昂、难以实施。因此,如何降低算法对预设阈值的依赖性,实现高效、自适应的异常判定,成为将该方法应用于实际运维场景的关键挑战。

3.4 基于极值理论的自适应阈值算法

3.4.1 经典 SPOT 算法原理与局限

SPOT 基于极值理论,其核心假设是:异常点(极值)出现在数据分布的尾部,并且尾部的行为可以用一种广义帕累托分布来近似。该方法特别适用于处理流数据,能够动态地更新对“正常”范围的理解。

经典 SPOT 算法为排除干扰,在更新模型时会剔除异常点。然而,在生产数据中,时序数据常存在概念漂移,即正常数据的整体分布会随时间缓慢变化。严格遵守经典方法会导致阈值衰减和误报率持续上升的问题,由于所有超过当前阈值的点都被剔除,用于更新 GPD 模型的数据池只包含了当前分布中较低的部分,用这个被“截断”的、仅包含低值的数据池重新拟合 GPD,会导致新估计出的阈值进一步降低,导致更多正常点被误判为异常并被排除在模型更新外,导致阈值降低,进入恶性循环。

3.4.2 包容性 SPOT 算法的改进策略

本研究对 SPOT 进行了关键改进:在更新模型时,将超过阈值的点也纳入训练数据池。此改进基于一项观察:在此数据场景下,超阈值点中仍然携带着关于当前数据范围的有价值信息。通过包容性学习,模型能动态跟踪分布变化,使阈值自适应调整,从而显著降低长期误报。

将 SPOT 与欧氏距离算法结合,即对欧氏距离计算后的多节点曲线平均距离序列应用 SPOT 算法,算法会根据一段初始的时间序列确定初始阈值,后续根据

曲线峰值点的分布重新拟合 GPD 模型，自动更新阈值，实现阈值的动态自适应。

3.4.3 集成学习框架设计与实现

集成学习通过结合多个基础模型的预测结果，以期获得比任何单一模型更稳定、更准确的最终判断。这种方法能有效降低因单一模型参数选择不当而导致的误报或漏报风险。在前期实验中发现，三种检测方法的检出率普遍优于准确率，表明单一模型更容易产生误报，故在集成学习中采用基于逻辑“与”的保守集成策略，即仅当所有基础模型均认为某个时间窗口异常时，才将其判定为最终异常。

对同一段时序数据，分别使用三种算法进行处理，再对检测结果进行融合。结果显示在阈值选择宽松的条件下，集成学习后仍达到了较高分数。结果说明即使某个模型的阈值设定不完美，其他模型的正确判断也能在一定程度上进行纠正，且三种算法可以从不同角度描述曲线相似性，集成方法能更全面地捕捉异常模式，使整体结果更稳定。

表 3-6 集成学习算法分数

算法	Precision	Recall	F1
欧氏距离 (threshold=3)	0.7251	1.0000	0.8406
AE (threshold=3)	0.6028	1.0000	0.7522
DBSCAN (eps=300)	0.3442	1.0000	0.5121
集成学习	0.9053	1.0000	0.9503

3.5 实验验证与对比分析

3.5.1 实验设置

本节实验在本文构建的单指标多节点数据集上进行，旨在验证所提出的改进算法的有效性。实验环境采用配备 Intel Core i7 处理器和 16GB 内存的服务器，算法实现基于 Python 3.8 和相关科学计算库。

实验评估采用异常检测领域通用的三个指标。精确率衡量检测结果中真实异常的比例，计算公式为 $Precision = TP / (TP + FP)$ ，其中 TP 表示正确检测的异常点数量， FP 表示误报的正常点数量。召回率衡量真实异常被正确检测的比例，计算公式为 $Recall = TP / (TP + FN)$ ，其中 FN 表示漏报的异常点数量。F1 分数是精确率和召回率的调和平均，计算公式为 $F1 = 2 \times Precision \times Recall / (Precision + Recall)$ ，用于综合评估检测效果。

实验对比的基线算法包括六种具有代表性的多元时序异常检测算法，涵盖基于重构、预测和生成对抗网络等主流范式。所有基线算法均使用其作者开源的原版代码和默认推荐参数进行训练与测试，以保证对比的公平性。

3.5.2 与基线算法实验结果对比

表3-2展示了各算法在测试集上的检测性能对比。从结果可以看出，传统的多元时序异常检测算法在单指标多节点场景下的表现普遍欠佳。TranAD 算法虽然取得了较高的精确率，但召回率仅为 0.3932，说明该方法漏报了大量异常。MAD_GAN 算法在召回率和精确率之间取得了相对均衡的结果，但 F1 分数仍不足 0.7。MTAD_GAT 算法的表现最差，其原因在于该算法主要针对指标间关联异常设计，难以有效捕捉负载敏感型指标的波动特性。

相比之下，基于曲线相似性的三种方法均取得了显著更优的检测效果。欧氏距离方法在最佳参数设置下达到了 0.9741 的 F1 分数，自编码器嵌入方法达到了 0.9691 的 F1 分数，DBSCAN 方法达到了 0.9632 的 F1 分数。这一结果充分验证了基于多节点曲线相似性进行异常检测的核心思路的有效性。通过对比节点间的曲线差异而非学习绝对的正常模式，能够有效规避负载波动带来的误报问题。

3.5.3 阈值稳定性与自适应效果分析

为验证改进的 SPOT 算法在阈值自适应方面的效果，本节将欧氏距离结合 SPOT 的方法与固定阈值方法进行对比。实验结果如表3-7所示。

表 3-7 SPOT 算法性能对比

算法	Precision	Recall	F1
欧氏距离 (最优固定阈值)	0.9826	0.9657	0.9741
欧氏距离 + SPOT	0.9679	0.9805	0.9742

从结果可以看出，结合 SPOT 算法的方法能够在无需人工调参的情况下达到与最优固定阈值方法相当的检测效果。更重要的是，SPOT 算法能够根据数据分布的变化动态调整阈值，在存在概念漂移的长期运行场景中具有更好的稳定性。改进的包容性 SPOT 算法通过将超阈值点纳入模型更新，有效避免了经典 SPOT 算法因阈值衰减导致的误报率上升问题。

集成学习方法通过融合多种检测算法的结果，进一步提升了检测的稳定性。如表3-6所示，即使在各基础算法采用宽松阈值设置的情况下，集成学习仍能达到 0.9503 的 F1 分数。这说明不同算法之间能够形成有效的互补，降低了对单一算法阈值选择的依赖。

3.6 本章小结

本章针对单指标多节点场景下的分布式集群异常检测问题开展了系统研究。首先基于联通大数据生产环境构建了高质量的单指标多节点时序数据集，为算法研究提供了数据支撑。其次提出了基于曲线相似性的异常检测方法，分别采用欧氏距离、自编码器嵌入和 DBSCAN 聚类三种方式度量节点间的曲线差异。针对单一方法阈值敏感的问题，设计了基于逻辑与策略的集成学习框架，并引入改

进的 SPOT 算法实现阈值的动态自适应调整。实验结果表明,所提方法在检测效果上显著优于现有的多元时序异常检测算法,验证了基于群体一致性思想的异常检测方法在分布式集群场景中的有效性。

第 4 章 数值-形状双视角融合的多指标多节点异常检测

4.1 集群多指标多节点数据集及异常检测算法框架

本节首先对所使用的多指标多节点 GPU 集群异常检测数据集进行介绍，在此基础上分析多指标多节点异常检测问题的关键挑战，并提出数值与形状双视角融合的异常检测算法整体框架。

4.1.1 多指标多节点数据集概述

本文所使用的数据集为阶跃星辰的 GPU 计算集群故障数据集，数据来源于实际生产环境中的 GPU 计算集群监控系统，该集群由数百台 GPU 计算节点组成，通过统一的资源调度系统执行深度学习训练任务。监控系统通过周期性采样的方式持续采集各节点各项运行状态数据，形成具有时间对齐关系的多节点多指标时间序列数据。

随着大规模 GPU 集群在人工智能训练、高性能计算及云计算场景中的广泛应用，集群运行过程中会产生大量高维度、多指标的监控时间序列数据。这些监控数据不仅反映了单个节点的运行状态，也隐含了节点之间的协同关系与整体运行规律。当集群中某个节点发生硬件故障、性能退化或资源异常时，其监控指标往往会表现出区别于其他节点的异常变化。因此，基于多指标多节点的时间序列数据开展异常检测研究，对于实现 GPU 集群的智能化运维和故障快速定位具有重要意义。

在 GPU 集群运行过程中，每个节点会产生多个维度的监控指标，这些指标从不同角度反映节点的运行状态和硬件健康状况。典型监控指标包括 GPU 核心温度 (gpu_temp)、GPU 功耗 (gpu_power_draw)、GPU 利用率 (gpu_utilization)、显存使用率 (gpu_memory_used)、GPU 频率 (gpu_clock)、节点 CPU 利用率以及系统负载等。这些指标共同构成节点在某一时间段内的运行特征向量。因此，对于一个包含 N 个节点、 M 个监控指标、时间长度为 T 的数据集，可以形式化表示为一个三维张量：

$$\mathbf{X} \in \mathbb{R}^{N \times M \times T} \quad (4-1)$$

在实际运行环境中，集群节点通常执行相同类型的计算任务，并通过调度系统实现负载均衡，因此在正常运行情况下，各节点在相同指标上的变化趋势通常具有较高的一致性。例如，在深度学习训练过程中，各 GPU 节点的功耗和温度变化往往呈现相似的波动模式。这种由系统调度机制和任务一致性所带来的节点间行为一致性，为异常检测提供了重要的判别依据。

当某个节点发生异常时，其监控指标曲线往往会偏离这种集群一致性模式。这种偏离可能表现为数值幅度上的异常，例如 GPU 温度显著高于其他节点；也

可能表现为变化模式上的异常，例如功耗曲线出现异常波动或抖动，而数值幅度仍处于正常范围内。因此，从多指标、多节点视角分析时间序列数据，能够更有效地区分节点个体异常与集群整体负载变化。

为评估异常检测算法的有效性，本文构建了包含 105 个真实故障案例的数据集。每个案例对应一次明确记录的 GPU 节点故障事件，故障类型包括 GPU 性能退化、温度异常升高、功耗异常波动等。在每个故障案例中，数据集均包含故障节点及其所在集群所有节点在故障发生时间窗口内的多指标时间序列数据。该数据集具有以下特点：

首先，数据来源于真实生产环境，具有较高的工程真实性和复杂性，不同节点之间存在一定程度的自然波动和噪声，使异常检测任务更加具有挑战性。

其次，数据具有典型的多指标特征，不同指标对异常的敏感程度不同。例如，温度指标对散热故障更敏感，而功耗指标对计算异常更敏感。因此，仅依赖单一指标难以全面刻画节点异常行为。

再次，数据具有显著的多节点关联特性，在正常运行状态下，大多数节点表现出一致的运行模式，而异常节点则表现为相对于其他节点的偏离。这种群体一致性为基于相似性的异常检测方法提供了理论基础。

最后，数据具有明显的时间序列特征，节点运行状态随时间动态变化，异常可能表现为短时突发异常，也可能表现为持续性异常。因此，异常检测方法需要能够同时捕捉时间维度变化和节点间差异。

基于上述数据特性，本文从数值特征和形状特征两个不同角度对节点异常行为进行建模，以提高异常检测的准确性和鲁棒性。

4.1.2 问题分析与算法框架

多指标多节点异常检测问题可形式化描述如下。给定一个包含 N 个节点、 M 个监控指标、时间长度为 T 的数据集，其形式为三维张量 $\mathbf{X} \in \mathbb{R}^{N \times M \times T}$ ，检测目标是识别在给定时间窗口内行为异常的节点。

该问题面临三个主要挑战。第一，不同指标对异常的敏感程度不同，需要设计合理的多指标融合策略。第二，节点异常可能表现为数值幅度的偏离，也可能表现为曲线形态的异常变化，单一视角难以全面覆盖。第三，大规模集群环境对算法效率提出了较高要求。

针对上述挑战，本章提出数值与形状双视角融合的异常检测框架。该框架的核心思想是从数值特征和形状特征两个互补视角分析节点异常行为，然后通过加权融合得到最终的异常判定结果。

在数值视角方面，采用基于欧氏距离的相似度度量方法，通过计算节点间时间序列的数值距离来量化差异程度，并采用加权平均策略融合多个指标的异常分数。在形状视角方面，采用改进的 SAX 符号化算法，通过将时间序列映射为符号序列并计算符号序列间的距离来捕捉曲线形态的差异。最终将两个视角的

异常分数进行加权融合,综合考虑数值偏离和形态变化两方面因素,输出节点的最终异常分数并进行排序以定位故障节点。

4.2 基于数值特征的异常检测算法

在分布式集群的性能监控中,节点的多个监控指标在正常状态下往往具有较强的相关性。当某个节点的行为出现异常时,其数值特征通常会偏离集群的整体行为模式。为了识别这种异常行为,本节提出了一种基于数值特征的异常检测算法,通过计算每个指标各节点之间的欧氏距离,结合加权融合设计和动态阈值策略,识别出异常节点。

在 GPU 集群中,节点的监控指标通常表现出一定的相关性。在训练任务正常运行时,各节点的资源使用、计算能力和负载情况应呈现出一定的一致性。当某个节点的行为出现异常时,其数值特征通常会偏离集群的整体行为模式。基于此,数值特征异常检测算法通过计算节点时间序列之间的数值差异,识别出行为异常的节点。在本节中,我们提出了一种基于数值特征的异常检测方法,该方法结合了欧氏距离度量、多指标加权融合以及 Z-score 标准化技术,以便更有效地识别异常节点。

4.2.1 多指标距离度量方法

在多指标多节点场景中,每个节点都有多个指标(如 CPU 利用率、内存使用率、网络带宽等)。为了比较不同节点之间的相似度,本研究使用了基于欧氏距离的度量方法。该方法可以计算节点之间在所有指标上的综合差异,从而有效捕捉节点异常的特征。

在分布式集群异常检测任务中,传统的单一指标异常检测方法往往无法全面揭示集群系统中的潜在问题,因为集群的健康状况是由多个指标共同决定的。因此,采用多指标距离度量方法能够更好地综合各个指标之间的关系,识别集群中的异常模式。

欧氏距离度量是常见的多指标距离计算方法之一,它通过计算各个节点之间的欧氏距离来量化它们的相似性。通过比较节点之间的距离,能够识别出那些偏离正常行为模式的节点。具体而言,首先将所有监控指标按时间窗口进行切分,并将每个时间窗口内的各个节点指标值转化为向量。然后,计算节点向量之间的欧氏距离,并根据距离矩阵生成每个节点的异常分数。节点的异常分数高,意味着该节点的行为与其他节点存在显著差异,从而被判定为异常。

此外,为了提升算法的鲁棒性和适用性,还可以通过加权距离度量来考虑不同指标对集群状态的不同影响。例如,某些关键指标如 CPU 负载或内存使用率对系统稳定性至关重要,而其他指标可能对集群性能的影响较小。因此,采用加权欧氏距离度量方法,给不同的监控指标赋予不同的权重,使得在计算距离时,能够更加突出重要指标的影响,从而提高异常检测的准确性。

4.2.2 加权融合设计

在多指标异常检测中，如何合理地将各个指标的异常信息融合成为一个全局异常评分是一个关键问题。常见的做法是采用加权融合策略，将每个指标的异常分数根据其重要性赋予不同的权重，再将所有指标的异常分数进行加权求和，从而得到每个节点的综合异常分数。

加权融合设计不仅考虑了每个指标本身的异常分数，还结合了指标的相对重要性。通过对每个指标分配不同的权重，可以使得那些在故障诊断中更具决定性的指标对最终的异常检测结果产生更大的影响。这种方式在多维度数据的异常检测中具有很大的优势，尤其是在分布式集群监控中，集群的健康状况往往是多个不同指标综合作用的结果。

具体的加权融合设计方法可以基于以下几个步骤：

(1) 确定指标权重：通过领域知识或者统计分析方法，确定各个监控指标的权重。这些权重可以根据指标在实际故障中的表现来动态调整。

(2) 计算单指标异常分数：对于每个时间窗口中的每个节点，基于各个监控指标的历史数据，计算该节点在各个指标上的异常分数。

(3) 加权求和：将每个指标的异常分数与其对应的权重相乘，得到加权后的异常分数，并将所有加权分数进行求和，得到该节点的最终异常分数。

(4) 阈值判定：根据加权后的异常分数，判断节点是否存在异常。可以采用设定阈值或者统计学方法（如基于分位数的阈值）来进行异常判定。

加权融合设计的优势在于，能够有效整合多维度的信息，避免单一指标异常检测可能带来的误报或漏报问题。尤其是在面对具有复杂关系的多集群系统时，采用加权融合能够提升整体的检测效果。

4.2.3 算法效率优化

由于分布式集群的规模较大，监控数据量庞大，直接对所有节点进行全量的计算将耗费大量的计算资源和时间。因此，提升异常检测算法的效率至关重要。在本研究中，针对欧氏距离计算和加权融合设计进行了以下几方面的优化，以提高算法的运行效率：

数据处理的优化：为了减少计算量，首先对监控数据进行了预处理，包括缺失值填充、数据标准化以及时间窗口的划分等。通过使用线性插值填充缺失值，确保数据的完整性和一致性。同时，通过对数据进行时间窗口切分，可以有效减少单次计算的规模，使得每次计算的窗口数据量相对较小，从而提高计算效率。

欧氏距离计算的优化：传统的欧氏距离计算需要对每对节点进行两两计算，这对于大规模集群来说会导致计算量急剧增加。因此，采用了向量化计算方法，并使用了 Scipy 库中的高效函数来计算节点间的欧氏距离矩阵。这样可以显著减少计算时间，提高算法的执行效率。

并行处理：在处理大量集群节点数据时，采用了批量处理和并行计算技术，将每个时间窗口的异常检测任务分配给不同的计算单元进行并行处理。这样能

够充分利用多核 CPU 资源，加速整个异常检测过程。

动态阈值判定：为了避免对所有数据都使用固定阈值进行判断，结合极值理论和统计学方法，采用了动态自适应阈值的方法。通过对历史数据进行分析，实时调整阈值，以适应不同集群的实际运行情况，从而进一步提高了检测的灵活性和准确性。

这些优化措施不仅减少了计算时间，也有效地降低了算法的内存占用，使得在大规模分布式集群环境下，异常检测算法能够高效运行。

4.3 基于形状特征的异常检测算法

4.3.1 经典 SAX 算法原理

SAX 是一种经典的时间序列符号化表示方法，其核心思想是将连续的时间序列转换为离散的符号序列，从而实现时间序列的降维和模式匹配。传统 SAX 算法主要包括四个步骤。

第一步是归一化处理。将原始时间序列标准化为均值为 0、标准差为 1 的标准正态分布，以消除量纲和基线差异。对于时间序列 $\{x_1, x_2, \dots, x_T\}$ ，归一化后的序列为：

$$\hat{x}_t = \frac{x_t - \mu}{\sigma} \quad (4-2)$$

其中 μ 和 σ 分别为序列的均值和标准差。

第二步是分段聚合近似。将归一化后的时间序列划分为 w 个等长的片段，并计算每个片段内所有数据点的平均值作为该片段的代表值。这一步骤实现了序列的降维，将长度为 T 的序列压缩为长度为 w 的表示。

第三步是符号化映射。基于标准正态分布的分位数点将数值空间划分为 α 个等概率的区间，其中 α 为字母表的大小。每个片段的平均值根据其所落入的区间被映射到对应的符号。通过这种方式，连续的时间序列被转换为由 α 个符号组成的符号序列。

第四步是距离计算。在符号空间中定义符号间的距离，进而计算符号序列之间的距离。传统 SAX 距离的定义中，相邻符号之间的距离为 0，非相邻符号之间的距离为对应分位数点之差。

4.3.2 基于 SAX 算法的符号化表示与模式匹配

为适应 GPU 集群故障检测场景的特殊需求，本文对传统 SAX 算法进行了两方面改进。

第一个改进是取消分段聚合步骤，保留原始时间分辨率。传统 SAX 的分段聚合虽然实现了降维，但可能丢失重要的细节信息。在 GPU 性能监控场景中，瞬时尖峰和短暂抖动往往包含重要的故障指示信息。因此，本文取消了分段处理，对归一化后的每个数据点直接进行符号化，使算法能够捕捉到更细微的时间点异常。

第二个改进是采用严格距离计算规则。传统 SAX 算法中，相邻符号被视为相似，其距离设为 0。在故障检测场景中，这种宽容性可能导致对形状变化的不敏感而造成漏报。本文修改距离计算规则，仅当两个符号完全相同时距离才为 0，否则计算实际距离：

$$d'(l_i, l_j) = \begin{cases} 0, & \text{if } i = j \\ \beta_{\max(i,j)-1} - \beta_{\min(i,j)}, & \text{if } i \neq j \end{cases} \quad (4-3)$$

其中 β_k 为第 k 个高斯分位数点。

基于改进的 SAX 算法，多节点形状异常检测的流程如下。首先对各节点的时间序列进行归一化处理，然后根据预设字母表大小计算高斯分位数点并将归一化序列映射为符号序列。接着计算所有节点对之间的符号序列距离，构建节点距离矩阵。最后计算每个节点与其他节点的平均距离，基于 Z-score 方法计算异常分数，并对各指标的异常分数进行加权聚合得到最终结果。

4.3.3 算法效率优化

为满足大规模 GPU 集群实时监控的需求，本文在算法实现层面进行了效率优化。

在距离矩阵计算方面，采用预计算符号距离表和向量化操作来避免运行时的重复计算。对于字母表大小为 α 的 SAX 表示，预先计算并存储 $\alpha \times \alpha$ 的符号距离矩阵，在运行时通过查表方式获取距离值。同时利用 NumPy 和 SciPy 库的向量化计算功能，将循环操作转换为矩阵运算，充分利用底层优化的线性代数库。

在数据处理方面，采用批量处理策略和内存优化技术。对于大规模数据集，按时间窗口进行分批处理，避免一次性加载全部数据导致的内存压力。同时采用稀疏矩阵表示和即时计算策略，仅在需要时计算特定节点对之间的距离，减少不必要的计算开销。

通过上述优化措施，算法在处理包含数百个节点、数千个时间点的多指标数据集时，单次检测耗时能够控制在秒级，满足实时监控的响应要求。

4.4 多算法融合与故障定位

4.4.1 加权融合设计

在多算法融合阶段，需要将基于数值特征和基于形状特征的检测结果进行综合。本文采用加权平均的融合策略，具体设计如下。

设节点 i 通过数值检测算法得到的异常分数为 s_i^{num} ，通过形状检测算法得到的异常分数为 s_i^{shape} 。由于两种算法输出的分数尺度可能不同，首先对各自的分数进行归一化处理，使其落入可比较的范围。然后通过加权平均得到融合后的异常分数：

$$s_i^{final} = w_{num} \cdot s_i^{num} + w_{shape} \cdot s_i^{shape} \quad (4-4)$$

其中 w_{num} 和 w_{shape} 分别为数值检测和形状检测的权重, 满足 $w_{num} + w_{shape} = 1$ 。

权重的设置可以基于领域知识或历史检测性能进行调整。在默认情况下, 本文采用等权重设置, 即 $w_{num} = w_{shape} = 0.5$ 。在实际应用中, 如果已知某类故障更多表现为数值异常, 可以适当提高数值检测的权重, 反之亦然。

对于多指标场景, 在单一视角内部也需要进行多指标融合。本文采用的策略是对各指标分别计算异常分数, 然后进行加权聚合。在加权聚合时, 遵循以下规则以提高鲁棒性。异常分数为 0 的指标不参与聚合计算, 因为这表明该指标在当前时间段内未能提供有效的异常信息。异常分数为负数的指标按 0 计算, 因为负的 Z-score 意味着该节点在该指标上表现正常甚至优于平均水平, 不对最终的异常判定产生负面影响。

4.4.2 故障定位与根因分析建议

在获得各节点的融合异常分数后, 根据分数对节点进行降序排序, 排名靠前的节点即为最可能发生故障的节点。在实际应用中, 通常取 Top-K 个节点作为故障候选进行进一步排查。

基于数值和形状双视角的检测结果, 还可以为故障类型判断提供参考。当节点表现出高数值异常分数且高形状异常分数时, 可能对应严重的硬件故障或系统级问题, 例如 GPU 核心损坏或散热系统失效。当节点表现出高数值异常分数但低形状异常分数时, 可能对应数值漂移或配置异常, 例如功耗阈值设置不当或资源分配失衡。当节点表现出低数值异常分数但高形状异常分数时, 可能对应间歇性故障或软件异常, 例如驱动程序问题或进程异常。

这种故障类型的初步判断能够为运维人员的后续排查工作提供方向性指导, 提高故障定位的效率。

4.5 实验验证与对比分析

4.5.1 实验设置

本节实验在阶跃星辰 GPU 计算集群故障数据集上进行。该数据集包含 105 个真实故障案例, 每个案例对应一次明确记录的 GPU 节点故障事件, 故障类型涵盖 GPU 性能退化、温度异常升高、功耗异常波动等。在每个故障案例中, 数据集包含故障节点及其所在集群所有节点在故障发生时间窗口内的多指标时间序列数据。

实验评估采用 Acc@5 和 Avg Rank 两个指标。Acc@5 表示真实故障机器出现在算法检测 Top-5 中的案例比例, 衡量算法的故障检测准确率。Avg Rank 为真实故障机器在 Top-5 中的平均排名, 衡量算法的故障定位效率, 数值越小表示定位越精准。

基线算法选择马氏距离方法, 该方法考虑了变量之间的协方差关系, 是多元异常检测领域的经典方法。

4.5.2 方法结果对比分析与案例研究

表4-1展示了各方法在 105 个故障案例上的检测效果对比。

表 4-1 多指标多节点异常检测方法对比

算法	Acc@5	Avg Rank
马氏距离	0.453	1.20
欧氏距离 + 加权聚合	0.800	1.25
改进 SAX+ 加权聚合	0.762	1.30
数值-形状融合	0.838	1.18

从结果可以看出,基于欧氏距离的数值检测方法在 Acc@5 指标上达到 0.800,显著优于马氏距离基线方法的 0.453。这表明在 GPU 集群场景中,基于节点间曲线相似性的检测思路比基于变量协方差关系的传统方法更为有效。

基于改进 SAX 的形状检测方法取得了 0.762 的 Acc@5,虽然略低于数值检测方法,但能够捕捉到一些数值方法遗漏的形态异常案例。将两种方法进行融合后,Acc@5 提升至 0.838, Avg Rank 降至 1.18,验证了双视角融合策略的有效性。

4.5.3 融合方法优势分析

融合方法相比单一视角方法具有两方面优势。

第一是更全面的异常覆盖。数值检测方法擅长识别幅度显著偏离的异常,而形状检测方法擅长识别变化模式异常但幅度正常的情况。通过融合两种视角,能够覆盖更多类型的异常模式。实验统计表明,约 15% 的故障案例主要表现为形态异常而非数值异常,单纯依赖数值方法会造成漏报。

第二是更稳定的检测性能。单一方法可能在特定类型的故障上表现优异但在其他类型上效果欠佳,而融合方法通过综合多个视角的判断结果,能够获得更稳定的整体性能,降低了对特定故障类型的依赖。

4.5.4 算法效率实验对比

为验证算法效率优化的效果,本节对比了优化前后的算法执行时间。实验选取包含 50 个节点、6 个监控指标、1000 个时间点的典型数据规模进行测试。

表 4-2 算法效率对比

算法	平均执行时间	加速比
欧氏距离 (优化前)	2.35s	1.0×
欧氏距离 (优化后)	0.28s	8.4×
改进 SAX (优化前)	5.12s	1.0×
改进 SAX (优化后)	0.45s	11.4×

通过向量化计算和预计算优化,欧氏距离方法的执行时间从 2.35 秒降至 0.28 秒,加速比达到 8.4 倍。改进 SAX 方法的执行时间从 5.12 秒降至 0.45 秒,加速比达到 11.4 倍。优化后的算法能够满足实时监控场景的响应要求。

4.6 本章小结

本章针对多指标多节点场景下的 GPU 集群异常检测问题,提出了数值与形状双视角融合的异常检测框架。在数值视角方面,采用基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面,提出了改进的 SAX 符号化表示算法,通过保留原始时间分辨率和采用严格距离计算规则来提升对形状变化的敏感性。最后通过加权融合两种视角的检测结果,实现了对不同类型异常的全面覆盖。在真实 GPU 集群故障数据集上的实验表明,所提融合方法的故障检测准确率达到 83.8%,显著优于单一视角方法和传统基线方法。

第5章 基于曲线相似性的异常检测系统

5.1 总体流程设计

基于曲线相似性的异常检测系统旨在为分布式集群提供实时的节点级异常监控能力。系统的总体流程分为数据采集、数据预处理、异常检测和结果展示四个阶段。

在数据采集阶段，系统通过对接集群的监控系统获取各节点的多维度时序数据。对于 YARN 集群，数据来源于 Prometheus 监控系统，通过其提供的 API 接口按固定时间间隔采集指标数据。对于 GPU 集群，数据来源于专用的 GPU 监控服务，采集 GPU 温度、功耗、利用率等核心指标。采集到的原始数据按节点和指标进行组织，形成时间对齐的多节点多指标时序数据结构。

在数据预处理阶段，系统对原始数据进行清洗和标准化处理。具体包括缺失值填充，采用线性插值方法处理因监控抖动造成的短时缺失。时间戳对齐，将不同数据源的采样点统一对齐到固定的时间间隔。数据归一化，对不同量纲的指标进行标准化处理以便后续计算。

在异常检测阶段，系统根据配置的算法类型执行相应的检测逻辑。对于单指标多节点场景，可选择欧氏距离、自编码器嵌入或 DBSCAN 方法，并支持集成学习和 SPOT 自适应阈值策略。对于多指标多节点场景，系统并行运行数值检测和形状检测两个模块，然后进行加权融合得到最终的异常分数。

在结果展示阶段，系统将检测结果以可视化的形式呈现给运维人员。主要展示内容包括异常节点列表及其异常分数排名，异常时间段的多节点曲线对比图，以及各指标的异常贡献度分析等。

5.2 系统架构与关键技术

系统采用分层架构设计，从下至上分为数据层、服务层和展示层三个层次。

数据层负责数据的存储和管理。原始监控数据存储与时序数据库中，支持高效的时间范围查询。检测结果和配置信息存储在关系型数据库中，支持灵活的查询和统计分析。

服务层是系统的核心，包含数据接入服务、预处理服务、检测引擎和结果输出服务四个主要组件。数据接入服务负责对接不同类型的监控数据源，提供统一的数据获取接口。预处理服务实现数据清洗、对齐和归一化等功能。检测引擎封装了本文提出的各种异常检测算法，支持灵活的算法配置和参数调整。结果输出服务将检测结果格式化后推送至展示层或外部系统。

展示层提供 Web 端的可视化界面，运维人员可以通过浏览器查看实时的检测结果和历史分析报告。界面采用响应式设计，支持在不同终端上访问。

系统的关键技术包括以下几个方面。在数据处理方面，采用流式处理架构支持实时数据接入和分析，避免了传统批处理模式的延迟问题。在算法调度方面，采用任务队列机制管理检测任务，支持多个集群的并发检测。在结果缓存方面，采用分级缓存策略减少重复计算，提高系统响应速度。

5.3 系统实现

系统采用 Python 作为主要开发语言，核心组件的实现细节如下。

数据接入服务通过 Prometheus 客户端库访问监控 API，支持 PromQL 查询语法灵活获取所需指标数据。对于其他数据源，通过适配器模式实现统一的数据接口，便于扩展支持新的监控系统。

预处理服务的核心是数据对齐和缺失值处理模块。数据对齐采用基于时间戳的重采样方法，将不规则采样的数据转换为固定间隔的时序数据。缺失值处理采用分段策略，对短时缺失采用线性插值，对长时间缺失的节点数据进行标记或剔除。

检测引擎采用模块化设计，每种算法封装为独立的检测器类，通过统一的接口进行调用。检测器的配置参数支持动态调整，无需重启服务即可生效。对于计算密集型的距离矩阵计算，采用 NumPy 和 SciPy 库进行向量化优化，并支持多进程并行加速。

结果输出服务将检测结果封装为标准的 JSON 格式，支持推送至消息队列、写入数据库或通过 WebSocket 实时推送至前端等多种输出方式。告警规则引擎根据配置的阈值和规则生成告警事件，并支持告警聚合和抑制策略。

5.4 系统应用效果展示

系统已在联通大数据生产底座平台上进行了部署验证。在实际应用中，系统对接了多个 YARN 集群的 Prometheus 监控数据，对 RPC Router 连接数等关键指标进行实时异常检测。

在性能方面，系统能够支持对包含数十个节点的集群进行秒级响应的异常检测。通过并行处理和缓存优化，多个集群的并发检测不会产生明显的性能瓶颈。

在检测效果方面，系统在生产环境中成功识别出多起节点级异常事件，包括因网络隔离导致的节点连接数异常波动、因资源竞争导致的单节点负载飙升等。相比传统的阈值告警方式，基于曲线相似性的检测方法能够有效区分集群整体负载变化和个别节点异常，显著降低了误报率。

在可视化展示方面，系统提供了多节点曲线对比图、异常分数热力图和告警趋势统计等多种视图，帮助运维人员快速定位和分析异常情况。异常事件的详情页面展示了相关节点在异常时段的各项指标变化，支持向下钻取进行深入分析。

5.5 本章小结

本章介绍了基于曲线相似性的异常检测系统的设计与实现。首先阐述了系统的总体流程，包括数据采集、预处理、异常检测和结果展示四个阶段。其次介绍了系统的分层架构和关键技术，涵盖数据层、服务层和展示层的功能设计。然后详细描述了各核心组件的实现细节，包括数据接入、预处理、检测引擎和结果输出等模块。最后展示了系统在联通大数据生产底座平台上的应用效果，验证了系统在实际生产环境中的可行性和有效性。

第6章 总结与展望

6.1 本文工作总结

分布式集群是支撑现代云计算与大数据处理的核心基础设施，其稳定性直接关系到上层业务的连续性。针对集群异常检测中现有方法未能有效利用多节点行为相似性这一重要先验知识的问题，本文提出了基于多元时序曲线相似性的分布式集群异常检测方法。本文的主要工作和贡献总结如下。

第一，本文提出了一种基于多节点曲线相似性的异常检测方法论。该方法突破了传统异常检测聚焦于时间与指标二维分析的框架，创新性地将集群节点这一空间维度系统性地引入模型，构建了时间、指标与节点的三维分析范式。通过利用分布式集群负载均衡机制所保证的节点行为相似性，将隐性的运维经验转化为显式的可计算先验知识，有效规避了因定义绝对正常模式而产生的误报问题。

第二，针对单指标多节点场景，本文基于联通大数据生产环境构建了高质量的时序数据集，并分别采用欧氏距离、自编码器嵌入和 DBSCAN 聚类三种方法度量节点曲线间的相似性。针对单一方法阈值敏感的问题，设计了基于逻辑与策略的集成学习框架，并引入改进的包容性 SPOT 算法实现阈值的动态自适应调整。实验结果表明，所提方法在 F1 分数上显著优于现有的多元时序异常检测算法。

第三，针对多指标多节点场景，本文提出了数值与形状双视角融合的异常检测框架。在数值视角方面，采用基于欧氏距离的多指标加权融合方法量化节点间的数值差异。在形状视角方面，提出了改进的 SAX 符号化表示算法，通过保留原始时间分辨率和采用严格距离计算规则来提升对形状变化的敏感性。在 GPU 集群故障数据集上的实验表明，融合方法的故障检测准确率达到 83.8%，显著优于单一视角方法和传统基线方法。

第四，本文设计并实现了基于曲线相似性的异常检测系统，实现了从数据采集、预处理、异常检测到结果展示的完整流程。系统已在联通大数据生产底座平台上进行部署验证，成功识别出多起节点级异常事件，验证了方法在实际生产环境中的可行性和有效性。

6.2 未来研究展望

虽然本文在基于多节点曲线相似性的异常检测方面取得了一定进展，但仍有若干值得深入研究的方向。

第一，更复杂的节点关系建模。本文假设同构集群内各节点地位平等，但实际系统中节点可能存在主从关系、依赖关系等复杂结构。未来可以引入图神经网络等方法显式建模节点间的拓扑关系，提升对复杂系统结构的刻画能力。

第二，异构集群的异常检测。本文主要关注同构集群中节点行为的相似性，但在异构集群中，不同类型节点的正常行为模式可能存在差异。未来可以研究如何在异构环境中识别可比较的节点组，并设计适应性更强的检测方法。

第三，在线增量学习能力。当前方法主要采用批量处理模式，对于持续运行的系统，需要能够在线更新模型以适应系统行为的长期演变。未来可以研究增量学习方法，使模型能够在保持对历史模式记忆的同时适应新的正常行为模式。

第四，异常根因分析。当前方法主要关注异常检测和故障节点定位，对于异常的具体原因缺乏深入分析。未来可以结合日志分析、配置变更追踪等信息，构建从异常检测到根因定位的完整链路，进一步提升系统的智能运维能力。

第五，跨集群知识迁移。不同集群的监控数据虽然在绝对值上可能存在差异，但其中蕴含的异常模式可能具有相似性。未来可以研究如何在多个集群之间进行知识迁移和经验共享，提升检测方法在新场景中的泛化能力。

参考文献

- [1] Su Y, Zhao Y, Niu C, et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network [C]//Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2019: 2828-2837.
- [2] Audibert J, Michiardi P, Guyard F, et al. Usad: Unsupervised anomaly detection on multivariate time series [C]//Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020: 3395-3404.
- [3] Deng A, Hooi B. Graph neural network-based anomaly detection in multivariate time series [C]//Proceedings of the AAAI Conference on Artificial Intelligence: volume 35. 2021: 4027-4035.
- [4] Tuli S, Casale G, Jennings N R. Tranad: Deep transformer networks for anomaly detection in multivariate time series data [C]//Proceedings of the VLDB Endowment: volume 15. 2022: 1201-1214.
- [5] Xu J, Wu H, Wang J, et al. Anomaly transformer: Time series anomaly detection with association discrepancy [J]. arXiv preprint arXiv:2110.02642, 2022.

附录一 附录中的公式

对公式的引用如，公式(附 1-1)

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad (\text{附 1-1})$$

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad (\text{附 1-2})$$

mathtext: $A, F, L, 2, 3, 5, \sigma$, mathnormal: $A, F, L, 2, 3, 5, \sigma$, mathrm: $A, F, L, 2, 3, 5, \sigma$.

mathbf: $\mathbf{A}, \mathbf{F}, \mathbf{L}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \boldsymbol{\sigma}$, mathit: $A, F, L, 2, 3, 5, \sigma$, mathsf: $A, F, L, 2, 3, 5, \sigma$.

mathtt: $A, F, L, 2, 3, 5, \sigma$, mathfrak: $\mathfrak{A}, \mathfrak{F}, \mathfrak{L}, 2, 3, 5, \sigma$, mathbb: $\mathbb{A}, \mathbb{F}, \mathbb{L}, 2, 3, 5, \sigma$.

mathcal: $\mathcal{A}, \mathcal{F}, \mathcal{L}, 2, 3, 5, \sigma$, mathscr: $\mathscr{A}, \mathscr{F}, \mathscr{L}, 2, 3, 5, \sigma$, boldsymbol: $\mathbf{A}, \mathbf{F}, \mathbf{L}, 2, 3, 5, \boldsymbol{\sigma}$.

vector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitvector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

matrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitmatrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

tensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unittensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

附录二 附录中的图表

附表测试

附表 2-1 这是一个样表

App Table 2-1 This is a sample table

行号	跨多列的标题							
Row 1	1	2	3	4	5	6	7	8

附表 2-2 这是一个样表

App Table 2-2 This is a sample table

行号	跨多列的标题							
Row 1	1	2	3	4	5	6	7	8

附图测试



附图 2-1 这是一个样图

App Figure 2-1 This is a sample figure

注: 对图片的注释

致 谢

2026 年 6 月

作者简历及攻读学位期间发表的学术论文与其他相关学术成果

作者简历：

已发表（或正式接受）的学术论文：

参加的研究项目及获奖情况：

